

C++ 三个月学习计划 · 细化版 + 每周打卡表

2 个文件 · 302 行 · 生成于 2026-10-08 19:08 · 行号对应源文件实际行号

00-计划细化版.md

Markdown

计划细化版（12 周 · 每天 1 小时）

定位不变：不背概念、不啃厚书、以代码实验为核心。细化做的是三件事—— ① 每周切到「当天看得见证据」的粒度；② 每周配可运行例题（见 code/ 与三份例题讲解）； ③ 每周留一个「回到 WebView 项目」的动作，避免学完不知道往哪用。

一、每日 1 小时的固定切法

时间	做什么	注意
10 min	复习昨天结论（凭记忆写三条，不许翻笔记）	写不出来就说明昨天只是"看过"
35 min	跑/改当天例题，先预测再运行	预测和实际不符的地方才是今天的收获
10 min	记录：结论一句话 + 反例一条 + 存疑一条	记到当日笔记，D7 汇总
5 min	把当天代码跑一遍 -fsanitize=address	让 ASan 变成习惯，不是排错时才用

每周固定动作（D6–D7）：把本周例题合成一个小题（自己加一个功能），然后跑 `bash code/run_all.sh` asan 全绿。

二、12 周路线图（周目标 + 例题 + 验收）

第一阶段：内存地基（第 1–2 周）

周	主题	例题	必须能回答的问题
1	对象大小 / 对齐 / 栈堆	p1_01_sizeof p1_02_layout p1_03_stack_heap	空类为什么是 1 字节？sizeof 为什么和成员顺序有关？栈为什么向低地址长？堆上的地址为什么和栈差很远？
2	vptr / 虚表 / 多态	p1_04_vptr p1_05_polymorphism	加 virtual 对象为什么变大 8 字节？同类对象为什么共用一张表？非虚函数为什么"看指针类型"？

第 1 周细化

- D1 空类与基础类型：先预测 `Empty/CharInt/IntChar` 的 `sizeof`，再跑 `p1_01`；关注 `HasFunction`（成员函数不占对象内存）。
- D2 对齐与排布：把 `LooseLayout` 和 `TightLayout` 的成员顺序互换，预测字节数；用 `offsetof` 验证。
- D3 字节可视化：自己写一个 `hexdump(const void*, size_t)`，把成员值手动标到字节序列上。
- D4 栈：在 `level()` 里再加两层递归，画出地址递减的图；解释"栈是复用"的证据。
- D5 堆与释放：把链表改成数组 `new Node[3]`，再改成 `new Node*[3]`，比较三次的地址和释放写法。
- D6 综合小题：写一个 `struct`，同时含 `char/int/double/指针/虚函数`，预测 `sizeof` 并解释每一字节。
- D7 复盘产出：一页笔记（可打印），三条结论 + 一个反例。

第 2 周细化

- D8 virtual 的代价：对同一个类加/去 virtual，记录 sizeof 变化与 vtable 地址。
- D9 vptr 位置：验证 vptr 在对象最前面（把对象首 8 字节当 void** 读，并与虚表函数地址对照）。
- D10 简单多态：基类指针指向派生类对象，比较 pb->speak() 与 pb->name()（虚 vs 非虚）。
- D11 单继承虚表：派生类重写 vs 新增虚函数，观察表槽位是"替换"还是"追加"。
- D12 虚析构：故意去掉 virtual ~Base()，用 -fsanitize=address 跑一次，看它报什么。
- D13 复盘：口述"对象内存布局 → vptr → 虚调用"的完整链条，写成一个 5 步流程图。
- D14 巩固：把 p1_05 改成三个派生类 + 一个工厂函数返回 unique_ptr<Base>（提前预热第 6 周）。

第二阶段：所有权与资源（第 3–6 周）

周	主题	例题	必须能回答的问题
3	string 底层	p2_01_string	SSO 的边界在哪？为什么 reserve 后地址不再变？size 和 capacity 差在哪？
4	vector 底层	p2_02_vector	扩容倍数与均摊复杂度？什么时候元素被"移动"而不是"拷贝"？迭代器为什么失效？
5	RAII	p2_03_raii	为什么异常路径也能释放？为什么禁止拷贝、只允许移动？析构顺序为什么是逆序？
6	智能指针	p2_04_unique_ptr p2_05_shared_ptr	move 后源指针为什么变空？循环引用为什么泄漏？weak_ptr 凭什么破坏？

每周的 7 天都按同一模板走：D1 观察现象 → D2 找出规则 → D3 看底层数据（容量/计数/地址）→ D4 复现一个坑 → D5 对照 Rust → D6 改造例题 → D7 复盘打印。

本周专属要求：

- 第 3 周：把 p2_01 的 SSO 边界从 22 改成"你自己的编译器实测值"，写清依据。
- 第 4 周：给 Tracked 去掉 noexcept 的移动构造，观察拷贝次数变化——这是"移动语义为什么要求 noexcept"的直接证据。
- 第 5 周：给 Buffer 加一个 swap，解释为什么移动赋值通常用 swap 实现。
- 第 6 周：把 p2_05 的循环引用用 ASan 跑一次，亲眼看它报 leak；再用 weak_ptr 修好。

第三阶段：多态边界 + 排坑 + 项目（第 7–12 周）

周	主题	例题	必须能回答的问题
7	多继承布局	p3_01_multi_inherit	为什么一个对象有两个 vptr？static_cast 和 reinterpret_cast 差在哪？
8	虚继承	p3_02_virtual_inherit	菱形继承为什么有两份基类？虚继承多出来的指针是什么？
9	内存错误现场	p3_03_overflow p3_04_use_after_free p3_05_double_free	ASan 三类报告的读法：越界 / 释放后使用 / 重复释放
10	排坑方法	p3_07_diagnose + 自造例题	不看调试器，靠地址与字节对比把问题缩小到一行
11–12	综合项目	p3_06_project	多态 + 容器 + 智能指针 + 文件 + RAII 如何组装成一个零泄漏程序

第 11–12 周的落地任务：把 p3_06_project 的 Student 换成你 WebView 项目里的真实领域对象（例如"文档/图层/任务"），保持同样的结构：

基类（纯虚接口 + 虚析构）

└ 派生类（不同算法/不同平台实现）

容器: `std::vector<std::unique_ptr<Base>>`

边界: 工厂函数负责创建, 返回 `unique_ptr` (所有权清晰)

持久化: `fstream` (RAII) 或平台文件 API 包一层 RAII

验证: ASan 全绿 + 退出码 0

三、工具速查（本机命令）

编译（带调试信息、关优化，便于看地址与变量）

```
clang++ -std=c++17 -g -O0 -Wall p1_04_vptr.cpp -o /tmp/demo
```

内存检查三件套

```
clang++ -std=c++17 -g -O0 -fsanitize=address demo.cpp -o demo_asan # 越界/UAF/重复释放
```

```
clang++ -std=c++17 -g -O0 -fsanitize=undefined demo.cpp -o demo_ubsan # 未定义行为
```

```
ASAN_OPTIONS=abort_on_error=0 ./demo_asan # macOS 上不支持 de
```

看符号、看大小、看段

```
nm -C demo | head # 列出符号 (-C 还原 C++ 名字)
```

```
size -m demo # 各段大小
```

```
otool -l demo | grep -A3 LC_MAIN
```

```
c++filt _Znam # 手写名字还原: operator new[]
```

调试

```
lldb ./demo # b main / r / frame variable / memory read -c 16 &obj
```

四、验收标准（每周都问自己）

1. 能预测: 看一段新代码, 能先说出 `sizeof` / 是否泄漏 / 会不会崩, 再运行验证。
2. 能复现: 每个结论都能用一个 20 行以内的小程序重现。
3. 能口述: 用"内存/所有权/生命周期"三个词讲清楚, 而不是背术语。
4. 能回收: 把结论落回 WebView 项目的一处具体代码 (哪怕只是加一个 RAII 守卫)。

五、反模式清单（出现就停下来）

- 只看不跑: 没有输出的"学习"不算学习。
- 只跑不改: 每题的"动手改一改"至少做一条。
- 用 ASan 当排错工具而不是日常习惯。
- 提前钻模板元编程、`constexpr` 细节——12 周计划里它们不是主线。
- 用 `reinterpret_cast` 掩盖类型问题 (例题 3-1 会演示它的后果)。
- 记住"结论"却说不出现象: 结论必须挂在你亲眼见过的输出上。

每周打卡表（12 周）

用法：每天 1 小时，做完打勾；「关键结论」一栏**用自己的话写**，写不出来说明当天只是看过。
每周 D7 把三件事汇总成一段：今天最反直觉的一条 + 一个反例 + 一个存疑点。

第 1 周 · 对象大小 / 对齐 / 栈堆

日	任务	完成 关键结论（自己写）
D1	例题 1-1：先预测 Empty/CharInt/IntChar 的 sizeof，再跑	☐
D2	例题 1-2：交换 Loose/Tight 成员顺序，用 offsetof 验证	☐
D3	自己写 hexdump()，把成员值标到字节序列上	☐
D4	例题 1-3：level() 再加两层，画地址递减图	☐
D5	例题 1-3：new Node[3] vs new Node*[3]，对比释放写法	☐
D6	综合：写一个含 char/int/double/指针/虚函数的 struct，逐字节解释	☐
D7	复盘：三条结论 + 一个反例，出打印稿	☐

验收：为什么 { double d; char c; } 是 16 不是 9？栈为什么向低地址长？

第 2 周 · vptr / 虚表 / 多态

日	任务	完成 关键结论（自己写）
D8	例题 1-4：加/去 virtual，记录 sizeof 与虚表地址	☐
D9	把对象首 8 字节当 void** 读，与虚表函数地址对照	☐
D10	例题 1-5：pb->speak() 与 pb->name() 为什么不同	☐
D11	派生类重写 vs 新增虚函数，看表槽位是替换还是追加	☐
D12	删掉 virtual ~Base()，用 ASan 跑一次看报错	☐
D13	口述「布局 → vptr → 虚调用」五步链条并写下来	☐
D14	三个派生类 + 工厂返回 unique_ptr<Base>（预热第 6 周）	☐

验收：加一个虚函数为什么让所有同类对象一起变大？非虚函数"看指针类型"是什么意思？

第 3 周 · string 底层

日	任务	完成 关键结论（自己写）
D15	例题 2-1：打印 size/capacity/data，判断数据在对象内还是堆上	☐
D16	实测 SS0 边界（本机 22 字符），写清判断依据	☐
D17	记录第 23 个字符时容量与地址各怎么跳	☐
D18	reserve(64) 后连续 append，对比地址是否变化	☐
D19	对照 Rust String：三件套、reserve、借用检查差别	☐
D20	改造例题：统计 1000 次 append 一共分配了几次	☐
D21	复盘 + 打印	☐

验收：s.c_str() 之后 append 一次，之前的指针为什么可能失效？

第 4 周 · vector 底层

日	任务	完成 关键结论 (自己写)
D22	例题 2-2: 记录完整容量序列, 验证 2 倍增长	☐
D23	观察地址"绕回来"现象, 说明为什么不能靠地址判断同一数组	☐
D24	Tracked 的拷贝/移动计数: 为什么是 拷贝=0 移动=7	☐
D25	去掉移动构造 noexcept, 对比计数变化	☐
D26	复现迭代器失效: 扩容/reserve 后旧指针作废	☐
D27	用 reserve 把多次分配压成一次, 记录前后差异	☐
D28	复盘 + 打印	☐

验收: 均摊 O(1) 是什么意思? 哪些操作会让迭代器失效?

第 5 周 · RAII

日	任务	完成 关键结论 (自己写)
D29	例题 2-3: 验证析构顺序是构造的逆序	☐
D30	异常路径: throw 后账本为什么仍然归零	☐
D31	禁止拷贝 + 实现移动, 说明各自解决什么问题	☐
D32	放进 vector 触发扩容, 确认只发生移动	☐
D33	加 swap, 用 swap 实现移动赋值	☐
D34	用 ASan 跑 RAII 版本, 确认零报错	☐
D35	复盘 + 打印	☐

验收: 为什么 `Buffer(const Buffer&) = delete;` 比"小心使用拷贝构造"更安全?

第 6 周 · 智能指针

日	任务	完成 关键结论 (自己写)
D36	例题 2-4: make_unique + 作用域自动释放	☐
D37	move 前后打印 <code>get()</code> , 解释源指针为什么变空	☐
D38	三种传参: 按值 (交所有权) / const 引用 (借用) / 移动	☐
D39	例题 2-5: 记录 <code>use_count</code> 的每一次变化	☐
D40	制造循环引用, 用 ASan 看泄漏报告	☐
D41	用 <code>weak_ptr</code> 破坏, 确认析构恢复	☐
D42	复盘 + 打印	☐

验收: 什么时候 `shared_ptr` 反而比 `unique_ptr` 危险?

第 7 周 · 多继承布局

日	任务	完成 关键结论 (自己写)
D43	例题 3-1: sizeof 与两个 <code>vptr</code> 的由来	☐
D44	打印 <code>pc/pa/pb</code> 三个地址, 量出偏移	☐

- D45 对比 `static_cast` / `reinterpret_cast` / `dynamic_cast` ☐
- D46 用基类指针 `delete`，验证虚析构的必要性 ☐
- D47 设计一个"多能力接口"组合（文件 / 剪贴板 / 窗口） ☐
- D48 把该接口用到你的项目类上，记录指针调整点 ☐
- D49 复盘 + 打印 ☐

验收：为什么不能把 `A*` 用 `reinterpret_cast` 转成 `B*`？

第 8 周 · 虚继承

日	任务	完成 关键结论（自己写）
D50	例题 3-2：菱形的两份 <code>Base</code> 数据，分别在哪个地址	☐
D51	写 <code>d.value</code> 触发二义性，记录编译器报错原文	☐
D52	改成虚继承，观察 <code>sizeof</code> 变化与唯一一份 <code>Base</code>	☐
D53	用 <code>dynamic_cast</code> 跨类转换，验证两条路径拿到同一地址	☐
D54	设计"共享上下文"（配置/日志）只保留一份	☐
D55	改造例题：去掉 <code>override</code> 看报错，理解重写约束	☐
D56	复盘 + 打印	☐

验收：虚继承多出来的那个指针是干什么用的？它和虚函数是什么关系？

第 9 周 · 内存错误现场（ASan）

日	任务	完成 关键结论（自己写）
D57	例题 3-3：堆越界写，读 ASan 报告第一行与 <code>allocated by</code>	☐
D58	例题 3-4：释放后使用，读 <code>freed by</code> 定位释放点	☐
D59	例题 3-5：重复释放，对比两次释放的调用栈行号	☐
D60	总结报告三要素：错误类型 / 地址 / 谁分配谁释放	☐
D61	修法练习：置空、RAII、 <code>unique_ptr</code> 各修一遍	☐
D62	给项目开着 ASan 跑一遍完整流程	☐
D63	复盘 + 打印	☐

验收：`heap-buffer-overflow` 与 `heap-use-after-free` 分别对应哪种代码动作？

第 10 周 · 排坑方法

日	任务	完成 关键结论（自己写）
D64	例题 3-7：对象切片、悬空引用、扩容失效三个现场	☐
D65	把每个现场压成 20 行以内的最小复现	☐
D66	列出你的"观测清单"（地址 / <code>sizeof</code> / <code>vptr</code> / <code>size/capacity</code> ）	☐
D67	用 <code>hexdump</code> 对比正常与异常路径的字节差异	☐
D68	自己造一个 bug 并只靠打印定位	☐
D69	加 <code>-fsanitize=undefined</code> 跑一遍，收集新发现	☐
D70	复盘 + 打印	☐

验收：为什么"对象切片"不崩溃却很危险？最快的发现信号是什么？

第 11 周 · 综合项目（上）

日	任务	完成 关键结论（自己写）
D71	跑通例题 3-6，通读三处设计（接口/所有权/持久化）	☐
D72	把 Student 换成你项目的真实领域对象	☐
D73	加 virtual std::string toJson() const	☐
D74	文件读写：批量导入 + 生成报告	☐
D75	ASan 全绿，退出码 0	☐
D76	写一页"内存视角说明"：谁在栈、谁在堆、谁拥有	☐
D77	复盘 + 打印	☐

验收：为什么 GradeBook 不需要自己写析构函数？

第 12 周 · 收口

日	任务	完成 关键结论（自己写）
D78	端到端跑通：宿主 C++ + Web 前端 + 数据读写	☐
D79	写清跨边界的所有权约定（字符串/数组/回调各一条）	☐
D80	边界用例：空数据、超大输入、异常路径	☐
D81	性能观察：reserve / move / 传参方式的实际差异	☐
D82	代码复盘：把所有 .get()、裸 new、reinterpret_cast 过一遍	☐
D83	输出三个月总结报告（含 12 周结论）	☐
D84	总复盘：这份计划哪几周要重做，下一阶段学什么	☐

验收：能不能用"内存 / 所有权 / 生命周期"三个词，讲清你项目里任意一个模块？

通用备注（每周可复制）

项目	记录
本周最反直觉的一条	
本周踩到的坑 + 最小复现	
存疑待查	
回到项目做了什么	